SAVETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES (SIMATS)

Programme:	B.E / B.Tech (Computer Science & Engineering)
Course Name & Code:	Software Testing (CSA3714) — CO3
Student Name & Register No:	Divyadharshini A (192411118)
Date of Submission:	Submission Date: 16-11-2026

INTRODUCTION

AgriMart System Context

AgriMart is a state-funded mobile marketplace application developed to connect farmers and buyers through a digital trading platform. The main objective of this application is to simplify agricultural trading by allowing farmers to register, upload their produce details, receive suggested market prices and sell their products directly to buyers.

The application is initially designed to support approximately 25,000 farmers and 3,000 buyers. Since the application is mainly intended for rural environments, it must operate effectively on low-cost Android smartphones and under limited internet connectivity. The system also supports regional language interfaces, including Tamil voice prompts, to help farmers with limited digital literacy.

As the Test Manager, the responsibility is to select appropriate testing methods, identify possible defects, prepare test plans, evaluate testing results and ensure that AgriMart satisfies its functional and non-functional requirements.

TASK 1: EVALUATE THE SCOPE OF UNIT TESTING

1.1 Definition of a Unit for AgriMart

Unit testing is the process of testing the smallest independently testable component of a software application. A unit may be a function, method, class or a small group of related methods that perform a specific operation.

The main purpose of unit testing is to verify whether each individual component performs its assigned functionality correctly without depending on the complete application.

In AgriMart, several modules contain important business rules that must be verified independently. For example, the Price Engine calculates the suggested selling price based on the market price and quality grade of agricultural products. Similarly, the Cart and Order module checks whether an order can be cancelled within the permitted time limit.

The following two examples represent suitable units in AgriMart.

Example 1: PriceCalculator Class in Module M3

The PriceCalculator class is responsible for calculating the suggested selling price of agricultural products.

The calculation depends on the base market price and the quality grade of the produce.

The quality multipliers are:

- Grade A = 1.10
- Grade B = 1.00
- Grade C = 0.85

For example, if the market price of tomatoes is ₹100 per kilogram and the selected quality is Grade A, the calculation will be:

Suggested Price = Base Price × Quality Multiplier

Suggested Price = 100 × 1.10

Suggested Price = ₹110.00

This calculation can be tested independently without connecting to the mobile application interface or the external Market-Price API.

1.2 Primary Goals of Unit Testing

Unit testing plays an important role in identifying defects during the early stages of software development.

The primary objectives are explained below.

1. Algorithmic Correctness

The first objective is to verify whether the algorithms and mathematical calculations produce accurate results.

In AgriMart, the Price Engine must apply the correct quality multipliers. If Grade C uses an incorrect multiplier of 0.80 instead of 0.85, the suggested price will be incorrect.

Unit testing helps identify such errors before the calculation is used in actual transactions.

2. Boundary Condition Verification

Boundary testing checks how a program behaves when inputs reach the minimum or maximum permitted values.

For example, AgriMart allows farmers to modify the suggested price within a range of ±15%.

If the suggested price is ₹100, the permitted range is ₹85 to ₹115.

The testing process must verify:

- A farmer price of ₹85 is accepted.

- A farmer price of ₹100 is accepted.
- A farmer price of ₹115 is accepted.
- A farmer price below ₹85 is rejected.
- A farmer price above ₹115 is rejected.

This ensures that the system correctly applies business restrictions.

3. Early Defect Identification

Unit testing helps identify errors before they become part of larger application workflows.

For example, if the PriceCalculator produces an incorrect amount, the developer can identify the issue directly within the calculation method rather than investigating the complete order and payment process.

This reduces debugging time and development cost.

4. Regression Protection

Regression testing checks whether previously working functionality continues to work after modifications.

Unit tests can be executed repeatedly whenever developers modify the application.

For example, when the Grade C multiplier changes from 0.80 to 0.85, existing tests can be executed again to verify that the updated calculation works correctly.

5. Improved Software Reliability

Unit testing improves the reliability of individual components by verifying normal inputs, invalid inputs, exceptional conditions and boundary values.

When all important units are tested independently, the probability of defects reaching integration and system testing is reduced.

1.3 Defects That Unit Testing Can and Cannot Identify

Unit testing is effective for identifying errors within isolated functions and classes. However, it cannot identify every type of software defect.

Defects That Unit Testing CAN Identify

1. Arithmetic calculation errors in the Price Engine.
2. Incorrect quality grade multipliers.
3. Boundary condition errors in farmer price validation.
4. Incorrect time calculations in order cancellation.
5. NullPointerExceptions during internal data processing.

6. Incorrect rounding of currency values.
7. Missing validation for negative or zero prices.
8. Incorrect conditional statements and logical operators.

For example, if the developer uses the greater-than operator instead of the greater-than-or-equal-to operator while checking the lower price limit, a valid boundary value may be rejected.

Unit testing can identify this problem by testing the exact boundary value.

Defects That Unit Testing CANNOT Identify

1. Incorrect HTTP headers between M3 and the external Market-Price API.
2. JSON schema mismatches between connected modules.
3. Network packet loss under rural 2G/3G connectivity.
4. Database deadlocks during simultaneous order placement.
5. Tamil font truncation on small mobile screens.
6. Payment transaction failures between M4 and M5.
7. Incorrect communication between logistics and order modules.
8. User interface problems involving buttons, screen layouts and navigation.

These defects require integration testing, system testing, compatibility testing, security testing or usability testing.

1.4 Critical Evaluation: Screen as Unit vs Function/Class as Unit

The development team proposed treating each application screen as an individual unit. However, this approach is not suitable for AgriMart's core business logic.

There are two possible approaches.

A. Screen as a Unit – Not Recommended

In this approach, an entire mobile application screen is considered a unit.

Testing includes:

- Android screen rendering.
- Button interactions.
- User input handling.
- Layout processing.
- Navigation between screens.
- Network communication.

Although screen testing is useful for validating user interface behavior, it creates several disadvantages when used as the primary definition of a unit.

First, screen-based testing is comparatively slow because it requires UI rendering and application lifecycle operations.

Second, the tests become fragile. Even a small layout modification may cause an existing test to fail although the actual business logic remains correct.

Third, defect identification becomes difficult because a failed screen test may result from a rendering issue, a button interaction problem or an incorrect calculation.

Therefore, screen-based testing should be performed at the UI and system testing levels rather than being treated as the strict unit-testing approach.

B. Function/Class as a Unit – Recommended

In this approach, individual functions and classes are tested independently.

For example:

- PriceCalculator tests pricing calculations.
- OrderCancellationGuard tests cancellation rules.
- PriceOverrideValidator checks farmer price limits.
- CurrencyFormatter verifies decimal precision.

This approach provides several advantages.

1. Tests execute quickly without requiring the complete mobile interface.
2. Defects can be identified within specific functions.
3. Test cases are easier to maintain.
4. Boundary conditions can be tested accurately.
5. Automated testing can be performed during continuous integration.
6. Developers can modify internal code without unnecessarily affecting unrelated tests.

1. Execute the test cases using JUnit.
2. Compare actual results with expected results.
3. Record failed test cases and identify defects.
4. Correct the defects and execute the tests again.
5. Generate the test execution and coverage reports.

1.6 Conclusion

Unit testing is essential for verifying the correctness of AgriMart's individual software components. It helps identify calculation errors, logical defects, invalid inputs and boundary condition problems at an early stage.

For AgriMart, function/class-level testing is the most suitable approach because the application contains important financial calculations and time-sensitive business rules.

Therefore, the development team should adopt function/class as the strict definition of a unit while performing separate UI testing for mobile screens. This approach improves test accuracy, reduces debugging effort and provides a reliable foundation for integration and system testing.

TASK 2: EVALUATE UNIT TEST PLANNING FOR MODULE M3

2.1 Introduction to Unit Test Planning

Unit test planning is the process of identifying the components to be tested, defining test objectives, preparing test cases, selecting testing tools and allocating resources before test execution.

A proper unit test plan ensures that all important functionalities are tested systematically.

In AgriMart, Module M3 is the Price Engine. It is one of the most critical modules because it calculates suggested prices based on daily market rates and agricultural product quality grades.

Incorrect price calculations can directly affect farmers and buyers. Therefore, M3 requires a detailed and carefully prepared unit test plan.

2.2 Objectives of the M3 Unit Test Plan

The major objectives are explained below.

1. Validate Grade-Based Price Calculation

The Price Engine must calculate suggested prices using the correct grade multipliers.

Quality Grade	Multiplier
Grade A	1.10
Grade B	1.00
Grade C	0.85

For a base market price of ₹100:

- Grade A = ₹110.
- Grade B = ₹100.
- Grade C = ₹85.

The testing process must verify that all three calculations produce accurate results.

2. Verify API Timeout Handling

The Market-Price API must respond within three seconds.

If the response takes longer than three seconds, the application should use cached market prices if the cached data is not older than 24 hours.

For example, if the API response takes 3.5 seconds and the cache is 12 hours old, the system should use the cached price.

3. Validate Cache Expiry

The system must not use cached market prices when their age exceeds 24 hours.

For example, if the API is unavailable and the cached price is 25 hours old, the system should block produce listing creation until valid pricing information becomes available.

4. Validate Farmer Price Overrides

Farmers are permitted to modify the suggested price within a range of ±15%.

The Price Engine must reject values outside this range.

5. Verify Currency Rounding

All calculated prices must be displayed with two decimal places.

For example, a calculated price of ₹110 must be displayed as ₹110.00.

Automated unit testing is recommended for M3 because the module contains mathematical calculations and several boundary conditions.

The following tools can be used.

JUnit 5: JUnit 5 is a Java testing framework used to create and execute automated unit test cases. It provides assertions to compare actual and expected results.

Mockito: Mockito is used to create mock objects and simulate external dependencies. It helps test M3 without connecting to the actual Market-Price API.

JaCoCo: JaCoCo is a Java code coverage tool used to identify which statements and branches have been executed during testing.

These tools allow the QA team to test M3 in a controlled environment.

2.5 Test Execution Procedure

1. Review the M3 functional requirements.
2. Identify all methods responsible for pricing and validation.
3. Prepare test data for Grades A, B and C.
4. Prepare boundary values for the ±15% override range.

5. Simulate API responses and timeout conditions using mock objects.
6. Execute test cases using JUnit 5.
7. Compare actual outputs with expected results.
8. Generate coverage reports using JaCoCo.
9. Record defects and execute failed tests again after correction.

2.6 Pass/Fail Criteria

The M3 testing process should satisfy the following criteria:

- All planned critical test cases must pass.
- All defined pricing calculations must produce correct values.
- The three-second API timeout rule must work correctly.
- Valid cached prices must be used only within the permitted 24-hour period.
- Invalid farmer price overrides must be rejected.
- No unhandled calculation exceptions should occur.
- Statement and branch coverage targets should be reviewed.

The original assignment proposes 100% statement and branch coverage for M3 pricing classes. This should be treated as the project's coverage target, along with requirement-based test verification.

2.7 Resource and Schedule Allocation

The original assignment allocates two QA Automation Engineers for two calendar days, representing 16 engineering hours.

Resource	**Responsibility**	**Allocation**
QA Engineer 1	Pricing and grade calculation tests	8 hours
QA Engineer 2	Timeout, cache and exception tests	8 hours
Testing Tools	JUnit 5, Mockito and JaCoCo	Throughout testing
Test Environment	Headless CI environment	Throughout testing

2.8 Critical Analysis: Why Unit Test Planning Is Often Skipped

Developers sometimes skip formal unit test planning because of project deadlines and pressure to complete features quickly.

Some developers assume that if the application works with a few sample inputs, it does not require additional testing.

However, this assumption creates serious risks.

1. Missing Boundary Conditions

Without proper planning, developers may test only normal inputs and ignore values at the exact three-second timeout or 24-hour cache boundary.

2. Financial Calculation Errors

An incorrect quality multiplier may produce inaccurate suggested prices and affect farmers financially.

3. Incomplete Test Coverage

Executing a function does not guarantee that all its decision branches have been verified.

4. Increased Debugging Cost

If defects are discovered during system testing, identifying the original source may require examining several connected modules.

5. Reduced Application Reliability

Without systematic planning, important exception conditions and invalid input scenarios may remain untested.

2.9 Conclusion

A formal unit test plan is essential for Module M3 because the Price Engine directly affects the financial accuracy of AgriMart.

The plan must cover grade-based calculations, API timeout behavior, cache validity, farmer price overrides and currency rounding.

Using JUnit 5, Mockito and JaCoCo will help the QA team perform repeatable automated testing and collect measurable evidence. Therefore, unit test planning should be considered an essential development activity rather than an optional task.

TASK 3: DESIGN AND EVALUATE UNIT TESTS FOR THE PRICE ENGINE

3.1 Introduction

The Price Engine (M3) is one of the most important modules in AgriMart. It calculates the suggested selling price of agricultural products using daily market rates and quality grade multipliers.

The module also handles API timeout conditions, cached market prices and farmer price modifications.

Since the calculated price directly influences agricultural transactions, the module must be tested carefully using positive testing, negative testing, boundary value analysis and exception testing.

3.2 Mathematical Derivations and Test Oracles

A test oracle is a reference used to determine whether the actual output of a program is correct.

For M3, the expected results are calculated using the approved business rules.

Quality Grade Multipliers

B. Negative Testing

Negative testing checks whether the application handles invalid inputs correctly.

Examples include:

- Negative base market price.
- Unknown quality grade.
- Missing market price.
- Invalid API response.
- Farmer price outside the permitted range.

The system should reject invalid data and provide controlled error handling.

C. Boundary Value Analysis

Boundary Value Analysis focuses on the values at and around the limits of a permitted range.

For example, the Grade A override range is ₹93.50 to ₹126.50.

The test should check:

- ₹93.49 – below the lower limit.
- ₹93.50 – exact lower limit.
- ₹93.51 – just above the lower limit.
- ₹126.49 – just below the upper limit.
- ₹126.50 – exact upper limit.
- ₹126.51 – above the upper limit.

This helps identify incorrect comparison operators and validation defects.

D. Exception Testing

Exception testing verifies whether the application handles unexpected situations without crashing.

Examples include:

- API returns HTTP 500.
- API returns invalid JSON.

- API response contains a negative price.
- Cached data is older than 24 hours.
- Required price information is missing.

The purpose of this test is to verify whether the Price Engine applies the correct Grade A multiplier.

Input:

- Base Price = ₹100.
- Grade = A.
- Multiplier = 1.10.

Expected Result = ₹110.00.

The test passes when the actual result matches the calculated expected price.

Test Case UT-M3-03: Grade C Calculation After Change C1

The original Grade C multiplier was 0.80. However, Change C1 updates it to 0.85.

Therefore, the new calculation must be:

₹100 × 0.85 = ₹85.00.

The test must verify that the updated multiplier is applied correctly.

Test Case UT-M3-05: Lower Override Boundary Failure

For Grade A, the minimum permitted price is ₹93.50.

If a farmer enters ₹93.39, the price is below the permitted range.

Expected Result: The application rejects the price and displays an appropriate validation message.

Test Case UT-M3-10: API Response Exactly at Three Seconds

The Market-Price API responds in exactly 3.000 seconds.

According to the supplied test specification, the system should use the live API price.

Expected Result: Live market price is utilized.

Test Case UT-M3-11: API Timeout with Valid Cache

The API response takes 3.001 seconds, exceeding the timeout threshold.

However, the cached market price is only 23 hours and 59 minutes old.

Expected Result: The system uses the cached price and identifies it as fallback data.

Test Case UT-M3-12: API Timeout with Stale Cache

The API response exceeds three seconds and the cached data is 24 hours and one minute old.

Since the cache has exceeded its permitted age, the application must not use it.

Expected Result: Produce listing creation is blocked and the user receives a price-unavailable message.

Test Case UT-M3-14: API Returns HTTP 500

The external Market-Price API returns an HTTP 500 server error.

However, a valid cached price is available and its age is only 12 hours.

Expected Result: The system uses the cached market price instead of failing the listing operation.

3.6 Test Execution Procedure

The following procedure should be followed while executing the M3 unit tests.

1. Prepare the PriceCalculator class and its required dependencies.
2. Configure the testing environment using JUnit 5.
3. Prepare input values for each quality grade.
4. Calculate expected prices independently.
5. Create mock API responses using Mockito or an appropriate stub.
6. Execute positive, negative and boundary test cases.
7. Compare actual results with expected results.
8. Record the pass/fail status of each test.
9. Correct the identified defects.
10. Execute the complete test suite again to verify the corrections.

3.7 Test Updates Required by Change C1

Change C1 modifies the Grade C multiplier from 0.80 to 0.85.

This change affects the expected calculation values.

Test Item	Old Value	Updated Value
Grade C multiplier	0.80	0.85
Suggested price	₹80.00	₹85.00
Lower override limit	₹68.00	₹72.25
Upper override limit	₹92.00	₹97.75

The testing team must update the following:

- Grade C calculation test.
- Grade C lower boundary test.
- Grade C upper boundary test.
- Test data files.
- Expected values in automated assertions.
- Dependent cart and payment test scenarios.

If the old test data is used after the modification, the testing system may report failures even when the updated implementation is correct.

3.8 Evaluation of Test Set Sufficiency

The test set should be evaluated using the following quality measures.

1. Statement Coverage

Statement coverage verifies whether all executable statements in the tested code have been executed.

2. Branch Coverage

Branch coverage verifies whether all possible decision outcomes have been executed.

For example, the price validation logic must be tested for both acceptance and rejection.

3. Boundary Completeness

The testing team should verify values below, at and above the important limits.

4. Mutation Testing

Mutation testing introduces small artificial changes into the source code to check whether the test cases can identify them.

The original assignment proposes a mutation score greater than 90% using the Pitest tool.

A strong mutation score indicates that the tests can detect many common implementation errors.

5. Requirement Traceability

Every important M3 requirement should be connected to one or more test cases. This ensures that no critical business rule is forgotten.

3.10 Conclusion

The Price Engine is a critical component of AgriMart because it determines the suggested selling price of agricultural products.

The designed test cases cover normal calculations, boundary conditions, invalid inputs, API timeout behavior, cache expiry and exception handling.

Change C1 must also be reflected in all relevant test data and expected results.

Therefore, a combination of positive testing, negative testing, Boundary Value Analysis, exception testing and automated coverage analysis provides a strong foundation for evaluating the correctness and reliability of Module M3.

TASK 4: DESIGN AND EVALUATE THE TEST HARNESS FOR MODULE M3

4.1 Introduction to Test Harness

A test harness is a collection of tools, test scripts, test data, drivers, stubs and supporting components used to execute software tests in a controlled environment.

It provides a structured method for executing test cases and comparing actual outputs with expected results.

In AgriMart, Module M3 depends on the external Market-Price API and crop information from Module M2. Testing M3 directly against these live dependencies can create unpredictable results because external services may be unavailable or delayed.

Therefore, a test harness is required to isolate M3 and provide controlled inputs and responses.

4.2 Main Components of the M3 Test Harness

A. Test Driver

A test driver is a program or test script that invokes the functions being tested.

In AgriMart, the JUnit test driver creates an instance of the PriceCalculator class, supplies test inputs and compares the actual output with the expected result.

For example, the driver can pass a base price of ₹100 and Grade A as inputs. It then checks whether the calculated result is ₹110.00.

The driver also executes different test cases automatically and records their results.

B. External Market-Price API Stub

A stub is a simulated component that replaces an external dependency during testing.

In M3, the external Market-Price API can be replaced with a WireMock-based HTTP stub.

The stub can be configured to return different responses.

Examples include:

1. HTTP 200 with a valid market price.
2. Delayed response exceeding three seconds.

3. HTTP 500 server error.
4. HTTP 503 service unavailable.
5. Malformed JSON response.
6. Missing price information.

This allows the testing team to verify different API conditions without depending on the actual external service.

C. Module M2 Stub

Module M2 manages agricultural produce listings and quality grades.

During M3 unit testing, the actual M2 database is not required.

An in-memory mock can provide predefined crop information such as:

- Crop Name: Tomatoes.
- Quality Grade: A.
- Product Quantity: 100 kg.
- Base Market Price: ₹100.

This ensures that M3 receives consistent inputs during testing.

D. Assertions and Test Reports

Assertions are used to compare actual results with expected results.

For example:

Expected Price = ₹110.00.

Actual Price = ₹110.00.

Result = PASS.

If the actual value is ₹105.00, the test fails and the difference is reported.

Test reports help developers identify failed scenarios and track corrections.

4.4 Step-by-Step Test Harness Execution

Step 1: Initialize the Testing Environment

The QA team starts the JUnit testing environment and loads the required test data.

Step 2: Create the PriceCalculator Object

An instance of the PriceCalculator class is created for executing the required methods.

Step 3: Configure Mock Dependencies

The Market-Price API stub and M2 stub are configured with predefined responses.

Step 4: Provide Test Inputs

The test driver supplies the base price, quality grade, API response time and cache timestamp.

Step 5: Execute the Test Method

The required M3 method is invoked using the supplied parameters.

Step 6: Validate the Output

The actual result is compared with the expected value using assertions.

Step 7: Generate the Test Report

The test execution results are recorded, including passed cases, failed cases and error details.

4.5 Reusable Test Data Management

Test data should be maintained separately from the test program.

For example, a JSON or CSV file can store:

- Test ID.
- Base market price.
- Quality grade.
- API response time.
- Cache age.
- Farmer override price.
- Expected output.

Using separate test data files provides several advantages.

1. New test cases can be added easily.
2. Existing test data can be reused.
3. Duplicate test code can be reduced.
4. Test data can be maintained in version control.
5. The same scenarios can be executed repeatedly.

Parameterized JUnit tests can be used to execute multiple test scenarios using the same test method.

4.6 Risks and Mitigation Techniques

One major risk associated with test harness design is called stub drift.

Stub drift occurs when the simulated API behavior becomes different from the actual external API behavior.

For example, the external Market-Price API may change a JSON field name from basePrice to another field name. If the WireMock stub is not updated, the tests may continue to pass even though the real API integration is broken.

The following mitigation techniques can be adopted.

1. Contract Testing

Contract testing verifies whether the application and external API follow the agreed request and response formats.

The Pact framework can be used to define and verify these contracts.

2. Regular Staging Verification

The testing team should periodically compare the stub behavior with the actual staging API.

3. Version Control

All stub configurations and test data should be maintained in a version control system.

4. Automated Test Execution

The test harness should execute automatically during continuous integration builds.

5. Documentation

All mock responses, assumptions and test configurations should be documented to ensure consistency across the testing team.

4.7 Advantages of the Proposed Test Harness

Advantage	Explanation
Isolation	M3 can be tested independently
Repeatability	The same inputs can be executed repeatedly
Faster testing	No need to depend on live external APIs
Error simulation	Different API failures can be reproduced
Better debugging	Defects can be identified more precisely
Reusability	Test data and mock configurations can be reused
Automation	Supports continuous integration testing

4.8 Evaluation of the Proposed Test Harness

The combination of JUnit 5, Mockito and WireMock provides a suitable testing environment for Module M3.

JUnit 5 controls test execution, Mockito simulates internal dependencies and WireMock simulates external HTTP services.

However, the testing team must ensure that simulated responses remain consistent with the real external API. Otherwise, the test results may provide misleading confidence.

Therefore, automated contract testing and periodic staging verification should be included.

4.9 Conclusion

A properly designed test harness is essential for testing AgriMart's Price Engine because M3 depends on external services and other application modules.

The use of test drivers, API stubs, mock objects and reusable test data helps create a controlled and repeatable testing environment.

JUnit 5, Mockito and WireMock are suitable tools for implementing this approach. By combining these tools with contract testing, the development team can identify defects early, reduce external dependencies and improve the reliability of Module M3.

5.1 Introduction to Integration Testing

Integration testing is a software testing technique used to verify whether two or more independently developed software modules communicate and function correctly when connected.

In unit testing, individual components are tested separately. However, in a real application, different modules must exchange information, share data and perform coordinated operations.

Integration testing identifies defects related to communication between modules, incorrect data exchange, interface mismatches and dependency failures.

In AgriMart, seven modules are developed to support the complete agricultural marketplace process. These modules are connected through different interfaces and external services.

For example, when a farmer registers successfully, the farmer's information must be transferred from Module M1 to Module M2. When the farmer lists agricultural produce, Module M3 must calculate the suggested selling price. Later, Module M4 uses this price during cart and order creation, while Module M5 processes the UPI payment.

If any module fails to communicate correctly with another module, the complete transaction may be affected.

Therefore, integration testing is necessary to ensure that all connected modules exchange information accurately.

5.3 Evaluation of Different Integration Testing Strategies

There are four major integration strategies that can be considered for AgriMart.

A. Big-Bang Integration Testing

Big-Bang Integration is a testing approach in which all software modules are integrated together at the same time after development is completed.

In this method, individual modules are developed and tested separately. After all modules are ready, they are connected and tested as a complete application.

Application in AgriMart:

Under this approach, the testing team would wait until all seven modules (M1–M7) are completed.

After completion, the team would integrate registration, produce listing, pricing, cart, payment, logistics and notifications together.

The complete transaction workflow would then be tested.

Advantages:

1. All modules are tested together in one environment.
2. It does not require extensive incremental integration planning.
3. It may be suitable for very small applications with limited dependencies.

Disadvantages:

1. Integration defects are identified very late.
2. Finding the actual source of an error becomes difficult.
3. A failure in one module may affect several connected modules.
4. The testing team must wait until all modules are completed.
5. Critical payment and pricing defects may remain undiscovered until the final stage.

Evaluation:

Big-Bang Integration is not suitable for AgriMart because M5 and M6 have not yet started development, while M4 and M7 are still in progress.

Waiting for all modules would delay integration testing and increase project risk.

Therefore, this strategy is rejected.

B. Top-Down Integration Testing

Top-Down Integration begins testing from the higher-level modules and gradually integrates lower-level modules.

In this approach, upper modules are tested first. If lower modules are unavailable, stubs are created to simulate their functionality.

A stub is a temporary replacement component that provides predefined responses to the calling module.

Application in AgriMart:

The testing team could begin with the user-facing workflows in M7 and M4.

Since M5 and M6 are not yet developed, stubs would be created to simulate UPI payment processing and logistics operations.

For example, when M4 submits an order to M5, a payment stub could return a successful transaction ID.

Advantages:

1. High-level workflows can be tested early.
2. User interaction and navigation defects can be identified.
3. Testing can begin before every lower-level module is completed.
4. It helps validate the overall control flow.

Disadvantages:

1. Many stubs may be required for incomplete lower-level modules.
2. Financial interface behavior may not match the actual implementation.
3. Maintaining several simulated dependencies increases testing effort.
4. Integration defects in lower-level modules may be identified later.

Evaluation:

Top-Down Integration is not the most efficient choice for AgriMart because several lower-level dependencies, particularly M5 and M6, are incomplete.

Creating numerous stubs would increase maintenance effort and delay the testing of important financial interfaces.

Therefore, this approach is rejected as the primary integration strategy.

C. Bottom-Up Integration Testing

Bottom-Up Integration begins with the lower-level modules and gradually integrates higher-level modules.

In this approach, completed lower-level components are tested first. Drivers may be used to simulate higher-level modules that are not yet available.

A driver is a temporary testing program that calls a lower-level module and supplies the required inputs.

Application in AgriMart:

The team can begin by integrating:

- M1 – Registration and KYC.
- M2 – Produce Listing.
- M3 – Price Engine.

These modules form the foundation of the agricultural marketplace.

After testing them, the team can gradually connect M4, M5 and M6.

Advantages:

1. Completed lower-level modules can be tested early.
2. Data processing and business logic defects can be identified.
3. It reduces dependence on incomplete upper-level modules.
4. It is suitable for testing pricing and registration functionality.

Disadvantages:

1. Higher-level user workflows are tested later.
2. Drivers may be required for unavailable upper-level components.
3. Complete transaction behavior cannot be verified immediately.
4. User interface and notification workflows may remain untested until later.

Evaluation:

Bottom-Up Integration is partially suitable for AgriMart because M1, M2 and M3 are already complete.

However, using only this approach would delay the testing of user-facing workflows and order processing.

Therefore, it is not selected as the complete integration strategy.

D. Sandwich or Hybrid Integration Testing

Sandwich Integration, also known as Hybrid Integration, combines Top-Down and Bottom-Up Integration approaches.

It allows the testing team to integrate completed lower-level modules while simultaneously testing available higher-level workflows.

This strategy reduces the limitations of using only one integration method.

Application in AgriMart:

The testing team should integrate the completed core business modules M1, M2 and M3 using a Bottom-Up approach.

At the same time, available higher-level order workflows in M4 can be connected using a Top-Down approach.

For example:

- M1 and M2 integration verifies farmer registration and produce listing.
- M2 and M3 integration verifies price calculation.
- M3 and M4 integration verifies cart pricing.
- M4 and the M5 payment stub verify checkout communication.

As M5 and M6 become available, their actual implementations can replace the temporary stubs.

Advantages:

1. Supports early integration of completed modules.
2. Allows user-facing workflows to be tested before the entire system is complete.
3. Reduces dependency on temporary stubs.
4. Helps identify interface defects earlier.
5. Supports parallel development and testing.
6. Provides better visibility into integration progress.

Disadvantages:

1. Requires careful coordination between testing teams.
2. Integration sequence must be properly planned.
3. Temporary stubs may still be required for unavailable external dependencies.
4. Test data and interface contracts must be maintained consistently.

5.4 Comparison of Integration Strategies

Evaluation Factor	**Big-Bang**	**Top-Down**	**Bottom-Up**	**Sandwich**
Testing starts	After all modules	Higher-level modules	Lower-level modules	Both levels
Stub requirement	Low initially	High	Low-level drivers	Controlled
Early defect detection	Poor	Good for upper modules	Good for lower modules	Good across levels
User workflow testing	Late	Early	Late	Early

Suitability for AgriMart	Low	Moderate	Moderate	High

5.5 Final Recommendation and Justification

After evaluating the four strategies, Sandwich Integration Testing is recommended for AgriMart.

The reason is that the project already has three completed modules, M1, M2 and M3, while M4 and M7 are still in development and M5 and M6 have not started.

A Sandwich approach allows the team to use completed modules immediately while continuing to validate available higher-level workflows.

It also helps identify critical financial and data exchange defects before the complete application is ready.

5.6 Conclusion

Integration testing plays an important role in verifying the communication and coordination between AgriMart's modules.

Among the four approaches, Big-Bang Integration creates late defect discovery, Top-Down Integration requires many stubs, and Bottom-Up Integration delays higher-level workflow testing.

Sandwich Integration provides a balanced approach by combining both top-down and bottom-up techniques.

Therefore, Sandwich Integration is the most appropriate strategy for AgriMart because it supports early defect identification, reduces testing delays and improves the reliability of module communication.

TASK 6: CONSTRUCT AND EVALUATE AN INTEGRATION TEST PLAN

6.1 Introduction to Integration Test Planning

An Integration Test Plan is a structured document that describes how connected software modules will be tested to verify their communication, data exchange and combined functionality.

It defines the testing sequence, interface requirements, test scenarios, expected outcomes, responsibilities and entry and exit conditions.

In AgriMart, integration testing is particularly important because the application contains several modules that depend on information from other modules.

For example, M3 depends on crop information from M2. M4 depends on the price calculated by M3. M5 depends on order information from M4.

If these modules exchange incorrect information, the application may display incorrect prices, create invalid orders or process incorrect payment amounts.

Therefore, the integration test plan must verify both successful and unsuccessful communication scenarios.

6.2 Objectives of the Integration Test Plan

The main objectives are:

1. Verify Module Communication

Ensure that connected modules communicate correctly through their defined interfaces.

2. Validate Data Accuracy

Verify that the data transferred between modules contains correct values and formats.

3. Identify Interface Defects

Identify incorrect API requests, missing fields, invalid responses and data conversion problems.

4. Verify Transaction Flow

Ensure that order creation and payment processing follow the correct sequence.

5. Validate Error Handling

Verify how the application behaves when an integrated module or external service fails.

6. Maintain Data Consistency

Ensure that changes in one module are correctly reflected in dependent modules.

6.3 Incremental Integration Execution Sequence

The following four integration steps are recommended for AgriMart.

Integration Step 1: M1 and M2 – Registration and Produce Listing

The first step integrates the Registration and KYC module with the Produce Listing module.

M1 is responsible for farmer registration and identity verification. M2 allows registered farmers to create produce listings.

The integration test must verify whether only farmers with valid KYC status can create listings.

Test Scenario:

A farmer completes registration and receives successful KYC verification.

The farmer then attempts to upload tomato produce details.

Expected Result:

- M1 provides the verified farmer identity.
- M2 accepts the farmer's registration reference.

- Produce listing is created successfully.
- The listing is associated with the correct farmer account.

Negative Scenario:

If KYC verification fails, M2 must prevent unauthorized produce listing creation.

Integration Step 2: M2 and M3 – Produce Listing and Price Engine

The second step integrates Produce Listing with the Price Engine.

M2 provides the crop name, quality grade and other product information. M3 uses the information to calculate the suggested market price.

Test Scenario:

A farmer lists Grade A tomatoes with a base market price of ₹100 per kilogram.

Expected Calculation:

₹100 × 1.10 = ₹110.00.

Expected Result:

- M2 sends the correct grade information to M3.
- M3 applies the Grade A multiplier.
- M3 returns ₹110.00.
- M2 displays the suggested price correctly.

The testing team must also verify Grade B and Grade C calculations.

Integration Step 3: M2 and M3 with M4 – Cart and Order

The third step integrates the produce information and price calculation modules with the Cart and Order module.

M4 retrieves produce details from M2 and suggested prices from M3.

Test Scenario:

A buyer selects 10 kg of Grade A tomatoes priced at ₹110 per kilogram.

Total Amount = 10 × ₹110.

Total Amount = ₹1,100.

Expected Result:

- M4 receives correct product information.
- M4 retrieves the correct price from M3.

- Cart subtotal is calculated accurately.
- Inventory availability is verified.
- The buyer can proceed to checkout.

The test should also verify the cached price scenario when the Market-Price API exceeds its three-second timeout.

Integration Step 4: M4 and M5 – Cart and UPI Payment

The fourth step integrates the Cart and Order module with the Payment module.

M4 submits the order details to M5. M5 communicates with the UPI payment gateway.

Test Scenario:

A buyer places an order worth ₹1,100 and selects UPI payment.

Expected Result:

- M4 sends the correct order amount to M5.
- M5 generates the UPI payment request.
- The payment gateway returns a unique transaction ID.
- Successful payment is recorded.
- Order status is updated correctly.
- Inventory is locked or adjusted according to the transaction rules.

Failure Scenario:

If the payment gateway returns an insufficient-funds error, the application should not mark the order as paid.

The order must return to the Pending Payment state.

6.5 Detailed Interface Test Cases

INT-01: Farmer Registration and Produce Listing

Objective: Verify the exchange of farmer identity information between M1 and M2.

Precondition: Farmer registration and KYC verification are completed.

Test Procedure:

1. Register a farmer in M1.
2. Complete KYC verification.
3. Obtain the verified farmer reference.

4. Open the Produce Listing module.
5. Submit a new crop listing.
6. Verify whether the listing is associated with the correct farmer.

Expected Result: Produce listing should be created successfully under the verified farmer account.

INT-03: Price Calculation Between M2 and M3

Objective: Verify that M3 receives correct crop grade information from M2.

Test Procedure:

1. Create a tomato listing in M2.
2. Select Grade A.
3. Send the product information to M3.
4. Calculate the suggested price.
5. Compare the actual result with the expected price.

Expected Result: The suggested price should be ₹110.00 for a base price of ₹100.

INT-05: Cart Pricing Between M3 and M4

Objective: Verify that M4 receives the correct suggested price from M3.

Test Procedure:

1. Select a Grade A tomato listing.
2. Retrieve its suggested price.
3. Add 10 kg to the cart.
4. Calculate the subtotal.
5. Compare the result with the expected amount.

Expected Result: The subtotal must be ₹1,100.

INT-08: Payment Failure Between M4 and M5

Objective: Verify the behavior when UPI payment fails.

Test Procedure:

1. Add produce to the buyer's cart.
2. Proceed to checkout.
3. Submit the payment request.

4. Simulate an insufficient-funds response.
5. Check the order status.
6. Verify that the order is not marked as paid.

Expected Result: The order must remain in Pending Payment status, and no successful payment confirmation should be recorded.

6.6 Entry and Exit Criteria

Entry and exit criteria define the conditions that must be satisfied before and after integration testing.

Entry Criteria

- Individual unit tests of the participating modules must pass.
- Required modules must be available in the integration environment.
- Interface specifications must be documented.
- Test data must be prepared.
- External API stubs must be configured where required.

Overall Exit All critical integration defects resolved

6.7 Risks in Integration Testing and Mitigation

1. API Communication Failure

An external API may be unavailable or may return unexpected responses.

Mitigation: Use mock services and validate API contracts.

2. Data Format Mismatch

One module may send information in a format that another module cannot process.

Mitigation: Use common data formats and interface validation.

3. Duplicate Transactions

Network failures may cause repeated payment requests.

Mitigation: Use unique transaction references and idempotency controls.

4. Incorrect Order Status

A payment may succeed while the order status remains unchanged.

Mitigation: Verify payment and order status consistency through transaction-level integration tests.

TASK 7: EVALUATE SYSTEM TESTING

7.1 Introduction to System Testing

System testing is the process of testing the complete and integrated software application to verify whether it satisfies the specified functional and non-functional requirements.

Unlike unit testing and integration testing, system testing evaluates the complete application as a single system.

In AgriMart, system testing must verify the complete agricultural marketplace workflow, starting from farmer registration and ending with payment confirmation and product pickup.

The testing process must also evaluate whether the application performs effectively under rural operating conditions.

These conditions include:

- Low-cost Android smartphones.
- Limited RAM and storage.
- Slow 2G/3G internet connections.
- Regional Tamil language interfaces.
- Limited digital literacy.
- High numbers of simultaneous users.
- External payment and logistics dependencies.

Therefore, the testing team must prioritize the system tests according to business importance and operational risk.

7.2 Objectives of System Testing

The major objectives are explained below.

1. Functional Verification

Verify whether all seven modules perform their required functions correctly when connected.

2. Performance Evaluation

Measure application response time, throughput, server stability and resource usage under expected and peak workloads.

3. Compatibility Verification

Verify whether the application works on low-cost Android devices and different supported screen sizes.

4. Security Evaluation

Check whether sensitive farmer information, KYC details and payment information are protected.

5. Usability Evaluation

Verify whether farmers can operate the application easily using regional language support and simple navigation.

6. Network Recovery Evaluation

Check whether the application handles poor connectivity, interrupted transactions and reconnection scenarios correctly.

7.3 System Testing Prioritization

Since AgriMart is designed for rural agricultural users, the testing team should focus on features that directly affect successful transactions and user adoption.

The following testing priorities are recommended.

A. Critical Priority

These tests must receive the highest attention because failures can directly affect business operations.

1. Performance and load testing.
2. Tamil language usability and accessibility testing.
3. End-to-end transaction testing.
4. Payment processing reliability.
5. Network interruption recovery.

B. High Priority

These tests are also essential for system reliability and security.

1. Low-end Android compatibility.
2. Payment security testing.
3. KYC information protection.
4. Database consistency.
5. Application recovery after network failure.

C. Reduced Priority

Some testing activities can receive lower priority because they are less important to the initial rural mobile deployment.

Examples include:

- High-end graphics rendering tests.
- Complex desktop browser compatibility.

- Advanced desktop-specific visual effects.

A. Performance and Load Testing

Performance testing evaluates the speed, responsiveness and stability of the application under different workload conditions.

AgriMart is expected to support approximately 25,000 farmers and 3,000 buyers during its initial launch season.

Therefore, the application must be tested under normal and peak usage conditions.

Test Scenario:

Simulate 5,000 concurrent order creation requests per minute under throttled 2G network conditions.

The testing environment should use:

- Network latency: 200 milliseconds.
- Bandwidth: 50 kbps.
- Concurrent order workload: 5,000 requests per minute.

Testing Procedure:

1. Configure the performance testing environment.
2. Prepare multiple virtual users.
3. Simulate simultaneous login and order creation requests.
4. Monitor server response time.
5. Measure transaction success and failure rates.
6. Monitor CPU, memory and database performance.
7. Record the results.

Expected Result:

The application should process the workload without server crashes, unacceptable response delays or unexplained transaction failures.

Tools such as Apache JMeter can be used to generate controlled workloads.

B. Usability and Accessibility Testing

Usability testing evaluates how easily users can understand and operate the application.

AgriMart is designed for farmers who may have limited digital literacy. Therefore, the interface should be simple, understandable and accessible.

Test Scenario:

A farmer who has limited reading ability attempts to list tomatoes using Tamil voice prompts.

Testing Procedure:

1. Select Tamil as the application language.
2. Open the produce listing screen.
3. Follow the voice instructions.
4. Select the crop and quality grade.
5. Enter the required quantity.
6. Review the suggested price.
7. Submit the listing.

Expected Result:

The farmer should be able to complete the listing process without requiring extensive technical assistance.

The testing team should also verify that:

- Tamil voice instructions are understandable.
- Audio playback works correctly.
- Important buttons are clearly visible.
- Instructions are short and meaningful.
- Text alternatives are available when audio cannot be played.

C. Compatibility Testing on Low-End Android Devices

Compatibility testing verifies whether the application functions correctly across different hardware and software environments.

Since AgriMart targets rural users, many users may operate low-cost smartphones with limited memory.

Test Environment:

- Device Category: Android Go.
- RAM: 1 GB.
- Screen Resolution: 480 × 800.
- Android API Level: 24.

Testing Procedure:

1. Install the application.

2. Complete farmer login.
3. Open the produce listing screen.
4. Upload crop details.
5. Navigate to the cart.
6. Perform checkout.
7. Verify notifications.
8. Monitor memory consumption and application crashes.

Expected Result:

The application should operate without unexpected crashes, screen overlapping or major functional failures.

D. Security Testing

Security testing verifies whether the application protects user information and prevents unauthorized access.

AgriMart handles sensitive farmer identity details, transaction records and UPI payment information.

Therefore, security testing must be performed on important interfaces.

Test Scenarios:

1. Attempt SQL injection through search inputs.
2. Attempt unauthorized access to farmer profiles.
3. Intercept payment callback requests.
4. Modify payment response values.
5. Check whether sensitive information is exposed in logs.
6. Verify authentication and session management.

Expected Result:

The application must reject unauthorized requests, validate payment responses and protect sensitive information.

Security testing tools and controlled test environments can be used to perform these checks.

E. Recovery and Network Drop Testing

Recovery testing evaluates whether the application can recover correctly after unexpected interruptions.

In rural areas, network connectivity may be unstable. A farmer may lose internet connectivity while submitting an order or completing a payment.

Test Scenario:

A buyer initiates a UPI payment, but the network connection is interrupted before the application receives the final response.

Testing Procedure:

1. Add produce to the cart.
2. Initiate UPI payment.
3. Disconnect the network during transaction submission.
4. Restore network connectivity.
5. Check the payment gateway transaction status.
6. Verify the order status.
7. Confirm that duplicate payments are not created.

Expected Result:

The application should reconcile the payment status correctly and avoid duplicate debits or lost orders.

7.6 Functional and Non-Functional System Testing

System testing covers two major categories.

Functional Testing

Functional testing verifies what the application does.

Examples:

- Farmer registration.
- KYC verification.

Non-Functional Testing

Non-functional testing verifies how well the application performs.

Examples:

- Application response time.
- Performance under heavy workload.
- Security and privacy.
- Usability.

- Device compatibility.
- Network recovery.
- Reliability and availability.

Both categories are necessary for the successful deployment of AgriMart.

7.7 Why Certain Testing Priorities Can Be Reduced

AgriMart is primarily designed as a mobile marketplace for rural users.

Therefore, the testing budget should be allocated according to actual user requirements.

High-end graphics rendering tests may be reduced because the application does not depend on advanced graphics functionality.

Complex desktop browser compatibility testing may also receive lower priority because the primary target platform is Android mobile devices.

However, reducing these priorities does not mean ignoring basic quality requirements. The application should still maintain acceptable visual consistency and supported platform compatibility.

7.8 System Testing Entry and Exit Criteria

Entry Criteria:

- Unit testing must be completed.
- Integration testing must be completed for the tested workflows.
- The integrated application must be available.
- Test devices and network configurations must be prepared.
- Test data must be available.

Exit Criteria:

- All critical system test cases must pass.
- No unresolved critical security defects should remain.
- Performance results must satisfy the defined requirements.
- Tamil voice navigation must work correctly.
- No unexplained payment duplication or transaction loss should occur.
- System testing reports must be documented.

TASK 8: EVALUATE REGRESSION TESTING FOR CHANGE

8.1 Introduction to Regression Testing

Regression testing is a software testing technique used to verify whether previously working application features continue to function correctly after modifications, bug fixes, enhancements or configuration changes.

During software development, changes are frequently introduced to improve existing functionality or correct identified defects.

However, one modification may unintentionally affect another connected feature.

Regression testing helps identify such unintended side effects.

In AgriMart, four important changes are introduced:

- C1: Grade C quality multiplier update in M3.
- C2: Order cancellation window reduction in M4.
- C3: New Tamil voice prompts in M7.
- C4: Duplicate farmer registration correction in M1.

These changes affect both the directly modified modules and their dependent modules.

Therefore, the testing team must perform impact analysis and select suitable regression test cases.

8.2 Importance of Regression Testing in AgriMart

Regression testing is especially important for AgriMart because the application handles financial transactions and connected business workflows.

For example, when the Grade C price multiplier changes, the suggested selling price also changes.

This modification can affect:

- Produce listing prices.
- Cart subtotals.
- Order totals.
- UPI payment amounts.
- Transaction records.

If the dependent modules are not tested again, buyers may be charged incorrect amounts.

Similarly, a modification to the order cancellation window may affect payment refund operations and logistics scheduling.

Therefore, regression testing must cover both direct and indirect dependencies.

8.3 Change Traceability and Impact Analysis

Change C1: Grade C Quality Multiplier Update in M3

Description:

The Grade C quality multiplier is updated from 0.80 to 0.85.

Previously:

Suggested Price = Base Price × 0.80.

After the change:

Suggested Price = Base Price × 0.85.

Example:

Assume the base market price is ₹100.

Old Grade C Price = ₹100 × 0.80 = ₹80.

Updated Grade C Price = ₹100 × 0.85 = ₹85.

Direct Impact:

M3 – Price Engine.

The calculation formula and expected output values must be updated.

Indirect Impact:

M4 – Cart and Order.

The cart subtotal may change because the suggested selling price has increased.

M5 – Payment.

The payment request amount must reflect the updated order total.

Required Regression Tests:

1. Verify Grade C suggested price.
2. Verify lower and upper farmer override limits.
3. Verify cart subtotal calculation.
4. Verify order total.
5. Verify UPI payment amount.

Direct Impact:

M4 – Cart and Order.

The cancellation timer and order state validation must be updated.

Indirect Impact:

M5 – Payment.

Refund processing may be triggered when an eligible order is cancelled.

M6 – Logistics and Pickup.

Pickup scheduling must be updated if an order is cancelled before dispatch.

Required Regression Tests:

1. Verify cancellation within the permitted window.
2. Verify cancellation after one hour.
3. Verify the exact boundary condition.
4. Verify payment refund processing.
5. Verify logistics dispatch cancellation.

Change C3: New Tamil Voice Prompts in M7

Description:

New Tamil voice prompts are added to improve accessibility for farmers.

These prompts provide audio instructions for registration, produce listing, order confirmation and other application operations.

Direct Impact:

M7 – Notification Module.

The audio playback functionality and voice prompt selection must be tested.

Indirect Impact:

Low-end mobile device performance and text notification fallback behavior may be affected.

The new audio files may consume additional storage and memory.

Required Regression Tests:

1. Verify Tamil voice prompt playback.
2. Verify correct prompt selection.
3. Verify audio playback on low-end devices.
4. Verify text notification fallback.
5. Verify that audio does not interrupt important application workflows.

Change C4: Duplicate Farmer Registration Correction in M1

Description:

A defect that allowed duplicate registration using the same mobile number is corrected.

The system must prevent multiple farmer accounts from being created with the same mobile number.

Direct Impact:

M1 – Registration and KYC.

The registration validation logic and mobile number uniqueness checks must be tested.

Indirect Impact:

M2 – Produce Listing.

Produce listings must remain associated with the correct farmer account.

The database unique key constraints must also be verified.

Required Regression Tests:

1. Register a farmer using a new mobile number.
2. Attempt registration using the same mobile number again.
3. Verify duplicate registration rejection.
4. Verify that the original farmer account remains accessible.
5. Verify that existing produce listings remain associated with the original account.

8.4 Change Impact Analysis Table

Change	Direct Impact	Indirect Impact	Main Regression Focus
C1	M3	M4, M5	Price and payment accuracy
C2	M4	M5, M6	Cancellation, refund and dispatch
C3	M7	Low-end device performance and text fallback	Tamil audio and accessibility
C4	M1	M2 and database constraints	Duplicate prevention and ownership

8.5 Regression Testing Strategy

The recommended approach is Selective Risk-Based Regression Testing.

In this strategy, the testing team does not execute every test case after every minor modification.

Instead, test cases are selected based on the affected modules, dependency relationships and business risks.

For AgriMart, the following approach is recommended.

A. Automated Regression Testing

The core sanity test suite should be automated using Appium and JUnit.

Appium can support automated mobile application testing, while JUnit can execute the relevant Java test suites.

The automated tests should verify:

- Farmer registration.
- Price calculations.
- Cart subtotal calculations.
- Order cancellation.
- Payment status.
- Tamil voice notification functionality.

These tests should execute continuously on every CI build.

B. Manual Regression Testing

Manual regression testing should focus on areas where human observation is important.

Examples include:

- Tamil voice clarity.
- Audio comprehension.
- Screen layout changes.
- Usability on low-cost devices.
- Accessibility and navigation behavior.

C. Impact-Based Testing

The testing team must identify all modules affected by each change.

For example, C1 directly modifies M3 but indirectly affects M4 and M5.

Step 8: Retest and Report

After corrections, failed test cases are executed again and the regression report is updated.

8.8 Benefits of Selective Risk-Based Regression Testing

1. Reduces unnecessary test execution.
2. Saves testing time and resources.

3. Focuses attention on high-risk business functionality.
4. Identifies defects in connected modules.
5. Protects existing application functionality.
6. Supports frequent software releases.
7. Improves confidence in modified builds.

8.9 Conclusion

Regression testing is essential for maintaining the reliability of AgriMart after software modifications.

The four changes C1–C4 affect different modules and their dependencies. Therefore, testing must cover both directly modified components and related workflows.

Selective Risk-Based Regression Testing is recommended because it prioritizes critical pricing, cancellation, payment, registration and notification functionality.

Automating the core sanity suite with Appium and JUnit, combined with manual verification of Tamil voice prompts and low-end device behavior, will help prevent existing functionality from breaking after updates.

PART 3 – ACCEPTANCE TESTING, OVERALL TESTING STRATEGY AND DESIGN CHALLENGE

TASK 9: EVALUATE ALPHA AND BETA

9.1 Introduction to Alpha and Beta Testing

Alpha and Beta testing are important stages of software testing performed before the final release of an application. These testing techniques help determine whether the software is ready for actual users and real-world environments.

Alpha testing is performed internally by the development and testing teams in a controlled environment. Its main purpose is to identify functional defects, usability problems, workflow errors and system stability issues before the application is released to external users.

Beta testing is performed by a selected group of actual users in their real operating environments. It helps evaluate the application's practical performance, user satisfaction, compatibility and reliability under real-world conditions.

In the AgriMart project, Alpha and Beta testing are particularly important because the application is designed for farmers and wholesale buyers who may have limited digital literacy and inconsistent internet connectivity.

The application must operate on low-cost Android smartphones, support Tamil voice navigation and provide reliable agricultural trading functionality.

Therefore, both Alpha and Beta testing are required to verify whether AgriMart is suitable for its intended users.

9.2 Alpha Testing Strategy for AgriMart

Alpha testing will be conducted in a controlled laboratory environment at the Agriculture Department Headquarters.

The testing process will involve 20 internal QA engineers and agricultural extension officers over a period of five days.

The primary objective is to verify system stability, workflow correctness and functional requirements before conducting field testing.

Alpha Testing Participants

- 20 internal QA engineers.
- Agricultural extension officers.
- Development team representatives.
- Software testing team members.

Alpha Testing Environment

The testing environment will contain:

- Android mobile devices.
- Stable Wi-Fi connectivity.
- Simulated 2G and 3G network conditions.
- Test farmer and buyer accounts.
- Mock payment gateway.
- Simulated agricultural market-price API.
- Test database containing sample produce information.

Alpha Testing Procedures

Step 1: Farmer Registration Testing

The QA team creates new farmer accounts and verifies mobile number validation, KYC verification and duplicate registration prevention.

Step 2: Produce Listing Testing

The testing team creates produce listings using different crop types and quality grades.

The team verifies whether produce information is stored correctly and associated with the appropriate farmer.

Step 3: Price Calculation Testing

The Price Engine is tested using Grade A, Grade B and Grade C produce.

The calculated prices are compared with expected values to identify arithmetic or business logic defects.

Step 4: Cart and Order Testing

The testing team adds agricultural products to the cart, verifies quantities and checks whether order totals are calculated correctly.

Step 5: Payment Testing

The payment workflow is tested using a simulated UPI gateway.

Successful payments, failed payments and interrupted payment requests are verified.

Step 6: Notification Testing

SMS notifications and Tamil voice prompts are checked to ensure that users receive correct information.

Step 7: Defect Reporting

All identified defects are documented, categorized according to severity and assigned to the development team for correction.

Expected Results of Alpha Testing

- Major functional defects are identified.
- Registration and KYC workflows operate correctly.
- Price calculations produce accurate results.
- Payment requests follow the correct transaction flow.
- Notification functionality is verified.
- The application is stable enough for controlled field deployment.

9.3 Beta Testing Strategy for AgriMart

Beta testing will be conducted after successful completion of Alpha testing.

Unlike Alpha testing, Beta testing takes place in real agricultural environments with actual farmers and wholesale buyers.

For AgriMart, Beta testing will be conducted across three rural farming districts.

The field deployment will involve:

- 200 real farmers.
- 30 wholesale buyers.
- Three rural farming districts.

- Two weeks of testing.

The purpose is to evaluate the application under realistic operating conditions.

Beta Testing Environment

The application will be installed on actual user devices.

Testing will include:

- Low-cost Android smartphones.
- Real 2G and 3G network connectivity.
- Tamil regional language interfaces.
- Realistic agricultural trading activities.
- Solar phone charging conditions.
- Actual produce listing and order workflows.

Beta Testing Procedures

Step 1: Field Deployment

The testing team installs AgriMart on selected farmers' and buyers' mobile devices.

Basic instructions are provided to help participants understand the application.

Step 2: Farmer Registration

Farmers register using their mobile numbers and complete the required verification process.

The testing team observes whether users can complete registration without technical assistance.

Step 3: Produce Listing

Farmers create listings for their agricultural products.

The team verifies whether users can select crops, enter quantities and understand the suggested selling prices.

Step 4: Buyer Activities

Wholesale buyers browse available products, compare prices and place orders.

The team evaluates the ease of product discovery and order creation.

Step 5: Network Testing

The application is tested under real rural network conditions.

The team observes how the application behaves when internet connectivity becomes slow or temporarily unavailable.

Step 6: Tamil Voice Navigation

Farmers use Tamil voice instructions to complete common activities.

The testing team collects feedback regarding audio clarity, pronunciation and ease of understanding.

Step 7: Solar Charging and Device Testing

The team observes application behavior on mobile devices used with solar charging facilities.

This helps identify practical usability and device limitations in rural areas.

Step 8: Feedback Collection

Farmers and buyers provide feedback about application functionality, accessibility and overall experience.

Day 14 Final feedback and evaluation Beta testing report prepared

9.4 Comparison Between Alpha and Beta Testing

Evaluation Factor	**Alpha Testing**	**Beta Testing**
Testing Location	Controlled laboratory	Real rural environment
Participants	20 internal QA engineers and extension officers	200 farmers and 30 buyers
Duration	5 days	2 weeks
Environment	Controlled and simulated	Actual operating conditions
Main Focus	Functional correctness and stability	Real-world usability and reliability
Network	Stable and simulated networks	Actual rural 2G/3G networks
Defect Identification	Technical and functional defects	Practical usage and environmental defects
Feedback	Internal testing reports	Actual user feedback

9.5 Feedback Collection Mechanism for Limited-Literacy Farmers

One of the major challenges in AgriMart is collecting meaningful feedback from farmers who may have limited reading and writing abilities.

Traditional feedback forms containing lengthy questions may be difficult for some users.

Therefore, the project should use simple and accessible feedback collection methods.

A. Single-Tap Voice Feedback

The application should provide a microphone button that allows farmers to record their feedback.

For example, after completing a produce listing, the application can ask the farmer to record a short voice message describing the experience.

This avoids the need to type long feedback messages.

B. Visual Star-Rating Emojis

Simple visual rating options can be displayed after important activities.

For example:

- Very Happy – Excellent experience.
- Happy – Good experience.
- Neutral – Average experience.
- Sad – Poor experience.
- Very Sad – Very poor experience.

Farmers can select an appropriate visual symbol without typing an explanation.

C. Agricultural Extension Officer Support

Field support extension officers can conduct structured weekly oral surveys.

They can ask farmers simple questions about:

- Ease of registration.
- Produce listing experience.
- Price calculation accuracy.
- Payment reliability.
- Tamil voice instructions.
- Overall application satisfaction.

The responses can be recorded and analyzed by the testing team.

9.6 Benefits of Alpha and Beta Testing

1. Helps identify defects before final deployment.
2. Improves application stability and reliability.
3. Validates real-world user requirements.
4. Evaluates performance under rural network conditions.
5. Improves Tamil language accessibility.

6. Collects valuable feedback from actual farmers.
7. Increases user confidence and satisfaction.

TASK 10: EVALUATE ACCEPTANCE TESTING

10.1 Introduction to Acceptance Testing

Acceptance testing is the final stage of software testing performed to determine whether a software application satisfies the business requirements and is ready for deployment.

It is generally conducted by customers, business representatives, end users or authorized stakeholders.

The main purpose of acceptance testing is to confirm that the developed application meets the agreed acceptance criteria.

In AgriMart, acceptance testing is important because the application is developed to support agricultural transactions between farmers and wholesale buyers.

The system must provide accurate pricing, reliable payment processing, secure farmer information and accessible regional language interfaces.

Before the application is officially released, the responsible stakeholders must verify whether these requirements have been successfully implemented.

10.2 Objectives of Acceptance Testing

The major objectives are:

1. Verify Business Requirements

Ensure that the application satisfies the requirements defined by the Agriculture Department and other stakeholders.

2. Validate Transaction Reliability

Verify that farmer and buyer transactions are completed successfully.

3. Ensure Financial Security

Confirm that payment processing does not introduce critical financial defects.

4. Verify Accessibility

Ensure that Tamil voice navigation works according to the required regional guidelines.

5. Validate Performance

Confirm that the system can support the expected number of simultaneous order requests.

6. Obtain Formal Approval

Collect formal acceptance from the authorized sign-off committee before deployment.

10.3 Measurable Acceptance Criteria

Acceptance criteria are specific conditions that must be satisfied for the software to be considered acceptable.

For AgriMart, four measurable acceptance criteria are defined.

Criterion 1: Transaction Completion Rate (CR1)

The system must achieve a 99.5% successful transaction completion rate under throttled 2G network conditions.

This criterion verifies the application's ability to complete transactions even when users experience slow internet connectivity.

Example:

If 10,000 transaction attempts are executed, at least 9,950 should complete successfully.

Calculation:

Successful Transaction Rate =

(Successful Transactions / Total Transaction Attempts) × 100

Required Rate = 99.5%.

Testing Procedure:

1. Configure the test environment with throttled 2G connectivity.
2. Generate 10,000 controlled transaction attempts.
3. Monitor completed and failed transactions.
4. Calculate the transaction success percentage.
5. Compare the result with the acceptance requirement.

Expected Result: The successful transaction completion rate must be at least 99.5%.

Criterion 2: Zero Critical or High-Severity Defects (CR2)

The application must contain zero unresolved critical or high-severity financial, security or data privacy defects.

This requirement is important because AgriMart handles farmer identity information and financial transactions.

Examples of critical defects include:

- Unauthorized access to farmer KYC information.
- Incorrect payment settlement.

- Duplicate payment deductions.
- Unauthorized modification of transaction amounts.
- Exposure of sensitive user information.

Testing Procedure:

1. Execute security test cases.
2. Verify payment transaction integrity.
3. Review access control mechanisms.
4. Inspect application logs for sensitive information exposure.
5. Classify identified defects according to severity.
6. Verify that all critical and high-severity defects are resolved before acceptance.

Expected Result: No unresolved critical or high-severity financial, security or privacy defects should remain.

Criterion 3: Tamil Voice Navigation Compliance (CR3)

All Tamil voice navigation flows must comply with the regional department guidelines.

This criterion ensures that farmers can understand and operate the application through Tamil voice instructions.

Testing Procedure:

1. Select Tamil as the application language.
2. Execute all supported voice navigation workflows.
3. Verify the pronunciation and clarity of prompts.
4. Test instructions on low-end mobile speakers.
5. Verify that users can complete important activities through the supported navigation flow.
6. Obtain feedback from regional representatives and participating farmers.

Expected Result: 100% compliance with the applicable regional department guidelines.

Criterion 4: Concurrent Order Request Performance (CR4)

The system must handle 5,000 concurrent order requests with a response time of less than three seconds.

This criterion evaluates the application's ability to process multiple requests simultaneously.

Testing Procedure:

1. Configure the performance testing environment.

2. Prepare 5,000 concurrent order requests.
3. Execute the requests using a load testing tool.
4. Measure response times.
5. Monitor server CPU, memory and database utilization.
6. Analyze successful and failed requests.

Expected Result: The system must handle the specified concurrent workload with response time below three seconds, according to the defined measurement conditions.

10.4 Acceptance Criteria Summary Table

Criterion ID	Acceptance Requirement	Measurement	Expected Result
CR1	Transaction completion	Success percentage under throttled 2G	At least 99.5%
CR2	Financial, security and privacy defects	Defect severity report	Zero unresolved critical/high defects
CR3	Tamil voice navigation	Regional compliance verification	100% compliance
CR4	Concurrent order requests	5,000 concurrent requests	Response time below 3 seconds

10.5 Acceptance Testing Execution Plan

Acceptance testing should follow a structured process to ensure that every requirement is verified.

Step 1: Requirement Review

The acceptance testing team reviews the functional and non-functional requirements.

Step 2: Acceptance Test Preparation

Test cases are prepared for all four acceptance criteria.

Step 3: Test Environment Setup

The required devices, network configurations, testing accounts and performance testing tools are prepared.

Step 4: Acceptance Test Execution

The testing team executes transaction, security, Tamil accessibility and performance tests.

Step 5: Result Evaluation

Actual results are compared with the defined acceptance criteria.

Step 6: Defect Resolution

Any identified critical or high-severity defects must be corrected and retested.

Step 7: Final Approval

The authorized sign-off committee reviews the testing evidence and provides the final decision.

10.6 Governance Sign-Off Framework

A formal sign-off process is necessary to ensure that AgriMart is approved by the responsible technical and business stakeholders.

The sign-off committee will consist of the following representatives.

A. Lead QA Test Manager

The Lead QA Test Manager is responsible for reviewing the complete testing results.

Responsibilities include:

- Reviewing test execution reports.
- Verifying acceptance criteria.
- Checking defect closure reports.
- Confirming regression testing completion.
- Providing the final quality assessment.

B. State Agriculture Department Technical Representative

The technical representative ensures that the application satisfies the department's technical and operational requirements.

Responsibilities include:

- Reviewing system functionality.
- Verifying technical compliance.
- Evaluating performance and security evidence.
- Confirming that the system meets deployment requirements.

C. Lead Farmer Cooperative Representative

The farmer cooperative representative provides the end-user perspective.

Responsibilities include:

- Reviewing farmer usability.

- Evaluating Tamil language accessibility.
- Verifying agricultural workflow suitability.
- Providing feedback about real-world operation.
- Confirming whether the application meets farmers' practical needs.

10.8 Importance of Acceptance Testing

Acceptance testing provides several benefits.

1. Confirms that business requirements are satisfied.
2. Reduces the risk of releasing incomplete software.
3. Protects users from serious financial and security defects.
4. Ensures accessibility for regional language users.
5. Validates the expected transaction performance.
6. Builds confidence among government stakeholders and farmers.
7. Provides formal evidence for deployment approval.

10.9 Conclusion

Acceptance testing is the final quality verification stage before AgriMart is released for actual use.

The four measurable acceptance criteria evaluate transaction reliability, security, Tamil voice accessibility and concurrent request performance.

The joint sign-off committee consisting of the Lead QA Test Manager, State Agriculture Department Technical Representative and Lead Farmer Cooperative Representative ensures that technical and business requirements are considered before deployment.

Therefore, acceptance testing provides a formal mechanism for determining whether AgriMart is reliable, secure and ready for real-world agricultural operations.

TASK 11: RECOMMEND AND DEFEND THE OVERALL TESTING

11.1 Introduction

An overall testing strategy is a structured approach that defines how different software testing activities will be planned, executed and evaluated throughout the software development lifecycle.

It combines multiple testing techniques to ensure that the application satisfies functional, technical and business requirements.

For AgriMart, selecting an appropriate testing strategy is important because the application contains seven interconnected modules and several external service dependencies.

The application must support 25,000 farmers and 3,000 buyers during its initial launch season.

It must also operate in rural environments where users may experience slow internet connections, low-end mobile hardware and regional language accessibility challenges.

Therefore, a combination of unit testing, integration testing, system testing, regression testing and field testing is required.

11.2 Recommended Multi-Level Testing Strategy

The recommended strategy for AgriMart consists of five major testing levels.

A. Unit Testing Using JUnit

Unit testing is the first level of the recommended testing strategy.

It verifies individual classes, functions and methods independently.

For AgriMart, special attention should be given to M3 Price Engine and M4 Order Cancellation logic.

Application:

- Verify Grade A, B and C price calculations.
- Validate the 3-second Market-Price API timeout logic.
- Verify the 24-hour cached price fallback.
- Test farmer price overrides within the ±15% threshold.
- Validate order cancellation time calculations.

Tool: JUnit 5.

Justification:

Unit testing identifies calculation and logical defects before modules are integrated.

It also provides rapid feedback to developers and reduces the cost of correcting defects.

B. Sandwich Integration Testing

Sandwich Integration Testing is recommended for connecting AgriMart's completed and developing modules.

It combines Top-Down and Bottom-Up Integration techniques.

Application:

- Integrate M1 with M2.
- Integrate M2 with M3.
- Connect M2 and M3 with M4.
- Connect M4 with the M5 payment gateway stub.

Justification:

This approach allows testing to begin before every module is completed.

It also helps identify communication and data exchange defects early, especially in pricing and order processing.

C. Risk-Based System Testing

Risk-Based System Testing prioritizes test activities according to the probability and impact of failure.

For AgriMart, the highest-risk areas are:

- Performance under heavy workload.
- UPI payment security.
- Rural network recovery.
- Tamil voice accessibility.
- Low-cost Android compatibility.

Application:

Performance testing should simulate the required concurrent order workload.

Usability testing should verify Tamil voice navigation with actual or representative users.

Recovery testing should verify that network interruptions do not cause duplicate payment deductions or lost orders.

Justification:

Risk-based testing ensures that the most important business functions receive sufficient testing attention.

It also helps allocate limited testing resources efficiently.

D. Automated Risk-Based Regression Testing

Regression testing verifies that existing features continue to work after software changes.

For AgriMart, the main changes are:

- C1: Grade C multiplier update.
- C2: Order cancellation window reduction.
- C3: New Tamil voice prompts.
- C4: Duplicate farmer registration correction.

Application:

Automated sanity tests should be executed on every CI build.

The regression suite should cover:

- Price calculations.
- Cart totals.
- Order cancellation.
- Payment processing.
- Farmer registration.
- Tamil notification functionality.

Tools: Appium and JUnit.

Justification:

Automated regression testing reduces repetitive manual work and identifies defects introduced by software modifications.

Manual testing should continue for Tamil voice clarity and other user-experience requirements.

E. Field Beta Testing

Field Beta testing evaluates AgriMart with actual farmers and wholesale buyers.

Application:

- Deploy the application across three rural farming districts.
- Involve 200 farmers and 30 wholesale buyers.
- Conduct field testing for two weeks.
- Evaluate 2G network performance.
- Verify Tamil voice comprehension.
- Observe actual agricultural trading workflows.

Justification:

Field testing provides practical evidence about whether the application works in its intended operating environment.

It also helps identify problems that may not appear in a controlled laboratory.

11.3 Overall Testing Strategy Comparison

Testing Level	Main Purpose	Recommended Tool or Method	Main Benefit

Unit Testing	Verify individual logic	JUnit 5	Early defect detection
Integration Testing	Verify module communication	Sandwich Integration	Reliable data exchange
System Testing	Verify complete application	JMeter and manual testing	Performance and usability
Regression Testing	Protect existing features	Appium and JUnit	Prevents repeated defects
Beta Testing	Verify real-world usage	Field deployment	Practical user validation

11.4 Resource and Schedule Allocation

The testing resources should be distributed according to the complexity and risk associated with each testing activity.

The recommended allocation is:

- 40% for Unit and Integration Testing.
- 35% for System and Performance Testing.
- 25% for Acceptance and Beta Testing.

The complete testing process will be conducted over a four-week execution timeline.

1. Early Defect Detection

Unit testing verifies critical calculations before integration.

This reduces the possibility of incorrect prices and order calculations reaching the final application.

2. Reduced Integration Risk

Sandwich Integration allows completed modules to be connected and tested early.

This reduces dependency on unfinished components.

3. Focus on Critical Business Requirements

Risk-based system testing prioritizes payment security, application performance and network reliability.

These areas directly affect the success of agricultural transactions.

4. Protection Against Software Changes

Automated regression testing verifies that modifications do not damage previously working functionality.

5. Real-World Validation

Field Beta testing provides direct feedback from farmers and wholesale buyers.

This ensures that the application is practical and accessible.

6. Efficient Resource Utilization

The proposed resource allocation distributes testing effort according to the importance of each testing stage.

11.6 Expected Benefits

The proposed strategy is expected to provide the following benefits:

- Improved software quality.
- Reduced integration defects.
- Accurate agricultural pricing.
- Reliable order and payment processing.
- Better compatibility with low-cost smartphones.
- Improved regional language accessibility.
- Reduced regression risks.
- Greater confidence before production deployment.

TASK 12: DAY-4 DESIGN CHALLENGE – STRATEGY ADAPTATION (8 MARKS)

12.1 Introduction to the Design Challenge

Software projects frequently experience changes in requirements, external dependencies and release schedules.

When such changes occur, the existing testing strategy may no longer be sufficient.

The testing manager must evaluate the impact of the changes and modify the testing process to maintain software quality.

In the AgriMart project, two important changes are introduced on Day 4:

1. Integration with the e-NAM agricultural trading platform.
2. An urgent hotfix release for the M5 Payment module within one week.

These changes introduce additional testing complexity and time constraints.

The e-NAM staging API is accessible for only two hours per day.

At the same time, the M5 hotfix must be tested and released within one week.

Therefore, the testing strategy must be redesigned to support continuous internal testing and rapid identification of critical defects.

12.2 Impact Analysis of e-NAM Integration

The e-NAM integration introduces an external trading API dependency into the AgriMart application.

e-NAM is an electronic agricultural trading platform that supports agricultural market transactions.

Integrating this platform may allow AgriMart to exchange relevant agricultural trading information with external systems.

However, external API integration introduces several testing challenges.

A. External API Dependency

AgriMart must communicate with the e-NAM staging API to exchange the required information.

The testing team must verify:

- API request formats.
- Authentication requirements.
- Response structures.
- Data validation.
- Error handling.
- Communication reliability.

B. Limited API Availability

The e-NAM staging API is accessible for only two hours per day.

This creates a serious testing limitation.

If the team depends entirely on the live staging API, developers and testers will have only a limited period to execute integration tests.

Any defect discovered outside this period may have to wait until the next available testing window.

This can delay defect correction and reduce development productivity.

C. Data Format and Compatibility Risks

The external platform may use specific data structures and field definitions.

If AgriMart sends an incorrect request format, the API may reject the request.

Similarly, an unexpected response format may cause application errors.

D. Authentication and Security Risks

External API authentication must be handled correctly.

Invalid credentials, expired tokens and unauthorized requests must be tested.

Sensitive authentication information must also be protected.

E. Error Handling and Recovery

The application must handle API failures without affecting unrelated AgriMart functionality.

For example, if the e-NAM API becomes temporarily unavailable, the application should provide an appropriate error message and avoid corrupting existing transaction records.

12.3 Impact Analysis of M5 Payment Hotfix

The second major change is an urgent hotfix for the M5 Payment module.

A hotfix is a software update released to correct an important defect or security problem without waiting for the next regular software release.

The M5 hotfix has a strict one-week release deadline.

Since M5 handles UPI payment settlement, the testing team must give it the highest priority.

Major Risks Associated with the Hotfix

1. Payment Settlement Failure

A change in payment processing logic may prevent successful transactions from being recorded.

2. Duplicate Payment

Repeated payment requests caused by network interruptions may result in duplicate deductions.

3. Incorrect Cart Status

The payment may succeed while the M4 order status remains unchanged.

4. Incorrect Pricing

Changes in payment processing may unintentionally affect the amount received from M3.

5. Transaction Atomicity Failure

A payment transaction may be completed without correctly updating the related order record.

The system must maintain consistency between payment and order states.

6. Regression Defects

A hotfix may unintentionally affect existing payment and order functionality.

Therefore, the modified payment module must undergo focused regression testing.

12.4 Combined Impact Analysis Table

Change	Main Impact	Testing Risk	Required Action

e-NAM Integration	External trading API communication	Limited API availability	API virtualization
e-NAM Integration	Request and response validation	Data compatibility defects	Contract testing
e-NAM Integration	Authentication	Unauthorized access	Security testing
M5 Hotfix	Payment settlement	Financial transaction failure	Critical-path regression
M5 Hotfix	M4 order status	Inconsistent transaction records	Cross-module testing
M5 Hotfix	M3 price rules	Incorrect payment amounts	Pricing regression

12.5 Strategy Adaptation Decision

Formal Decision: MODIFY AND REDESIGN THE TESTING STRATEGY.

The existing testing strategy should be modified to accommodate the limited e-NAM API availability and the urgent M5 hotfix deadline.

The testing team should introduce two major actions:

1. Build an in-house mock e-NAM API virtualization stub.
2. Execute a focused one-day automated critical-path regression suite for the M5 hotfix.

These changes will allow internal testing to continue without depending entirely on the limited live API availability.

12.6 Solution 1: e-NAM API Simulation Using Virtualization Stubs

API virtualization is a testing technique in which a simulated service reproduces the behavior of an external API.

It allows developers and testers to execute API-related testing even when the actual external service is unavailable.

For AgriMart, an in-house mock e-NAM API should be developed using the official API specifications.

The mock service should reproduce the expected request and response behavior.

A. Purpose of the Mock API

The mock e-NAM API will allow continuous internal development and testing outside the two-hour live staging window.

Instead of waiting for the external API to become available, developers can use the simulated service for most development and functional testing activities.

B. Mock API Features

The virtualization stub should support the following scenarios.

1. Successful API Response

The mock service returns a valid response when AgriMart sends a correct request.

2. Invalid Request

The mock service returns an appropriate error when required fields are missing or incorrect.

3. Authentication Failure

The mock service simulates unauthorized access when invalid authentication information is provided.

4. Server Error

The mock service returns HTTP 500 or 503 responses to simulate temporary external service failures.

5. Delayed Response

The mock service introduces response delays to verify timeout and retry handling.

6. Malformed Response

The mock service returns invalid or incomplete response data to verify error handling.

C. Testing Procedure

1. Review the official e-NAM API specifications.
2. Identify required endpoints and data structures.
3. Develop an in-house mock service.
4. Configure successful and unsuccessful response scenarios.
5. Connect the AgriMart integration module to the mock service.
6. Execute API request and response validation.
7. Verify authentication and error handling.
8. Record the results.
9. Use the live staging API during its available two-hour period to perform real contract and integration verification.

D. Benefits of API Virtualization

- Enables continuous internal testing.

- Reduces dependency on external API availability.
- Supports repeatable test execution.
- Allows controlled error simulation.
- Helps identify API integration defects early.
- Improves development productivity.
- Reduces delays caused by limited staging access.

12.7 Solution 2: Focused One-Day Automated Regression for M5 Hotfix

The second action is to execute a focused one-day automated critical-path regression suite.

Since the hotfix must be released within one week, the testing team should avoid unnecessary testing activities that do not directly affect the modified transaction workflow.

The regression suite should prioritize three critical modules:

- M5 – Payment Settlement.
- M4 – Cart and Order Status.
- M3 – Price Engine Rules.

A. M5 Payment Settlement Testing

The testing team must verify that the modified payment module correctly processes successful and unsuccessful UPI transactions.

Test scenarios include:

1. Successful payment.
2. Failed payment.
3. Duplicate payment request.
4. Payment timeout.
5. Payment callback verification.
6. Transaction rollback.

Expected Result: Payment transactions must be recorded correctly without duplicate deductions or incorrect settlement.

B. M4 Cart Status Testing

The testing team must verify whether the cart and order status are updated correctly after payment processing.

Test scenarios include:

1. Successful payment updates the order status.
2. Failed payment does not mark the order as paid.
3. Interrupted payment is reconciled correctly.
4. Repeated payment requests do not create duplicate orders.
5. Eligible cancellation updates the order status correctly.

Expected Result: Order records must remain consistent with the actual payment status.

C. M3 Price Engine Rule Testing

The Price Engine must also be tested because the payment amount depends on the calculated product price.

Test scenarios include:

1. Grade A price calculation.
2. Grade B price calculation.
3. Grade C price calculation.
4. Farmer price override validation.
5. Cached price fallback.
6. Updated Grade C multiplier verification.

Expected Result: The payment amount must match the correct calculated order amount.

12.10 CI Automation and Continuous Regression

Continuous Integration (CI) is a software development practice in which code changes are automatically built and tested.

For AgriMart, the automated regression suite should be connected to the CI pipeline.

Whenever a new payment-related code change is submitted:

1. The CI system builds the application.
2. Automated unit tests are executed.
3. Payment regression tests are executed.
4. Cart and order consistency tests are executed.
5. Price Engine tests are executed.
6. Failed tests are reported to developers.

7. A build is approved for further testing only when the required checks pass.

The automated suite should remain available continuously, allowing the team to run tests throughout the development and hotfix period.

12.11 Evidence-Based Defense of the Modified Strategy

The modified strategy is more suitable than depending exclusively on the live e-NAM staging API.

The e-NAM staging API is accessible for only two hours per day.

If the testing team relies entirely on the external service, the remaining 22 hours cannot be used for live integration testing.

This creates testing bottlenecks and increases the possibility of missing defects before the deadline.

By developing an in-house API virtualization stub, the team can perform continuous internal testing using controlled responses.

The live two-hour window can then be used for essential contract verification and real integration checks.

Similarly, the M5 hotfix introduces a serious financial risk.

A defect in payment settlement may affect buyers, farmers and transaction records.

Therefore, a focused one-day automated regression suite should prioritize M5 payment settlement, M4 order status and M3 price rules.

This strategy concentrates testing effort on the most critical transaction pathway.

Comparison of Original and Modified Strategies

Evaluation Factor	**Original Strategy**	**Modified Strategy**
External API Testing	Depends heavily on live e-NAM API	Uses virtualization stub
API Availability	Limited to 2 hours per day	Continuous internal testing
Hotfix Testing	General regression approach	Focused one-day critical-path suite
Payment Verification	Standard module testing	Priority M5 settlement testing
Cross-Module Testing	General integration testing	M3–M4–M5 critical-path testing
Defect Detection	May be delayed by external dependency	Earlier controlled testing
Deadline Management	Higher delay risk	Better control of one-week deadline

12.12 Expected Outcomes of the Modified Strategy

The modified testing strategy is expected to provide the following outcomes:

1. Continuous internal testing of e-NAM integration.
2. Reduced dependence on the limited live API window.
3. Early identification of API request and response defects.
4. Faster verification of payment-related changes.
5. Improved reliability of M5 payment settlement.
6. Better consistency between cart, pricing and payment modules.
7. Reduced regression risk during the hotfix release.
8. Better control over the one-week release deadline.

The strategy is designed to reduce financial and integration risks. However, successful outcomes must be confirmed through actual test evidence rather than assumed in advance.